

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Arya Arrozza Ridho Syaputra

NIM. 2410817210010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arya Arrozza Ridho Syaputra
NIM : 2410817210010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL	7
A. Source Code	10
B. Output Program.....	51
C. Pembahasan.....	59
D. Essay	71
E. Daftar Pustaka	72
Tautan GIT	73

DAFTAR GAMBAR

Gambar 1. Contoh UI List.....	8
Gambar 2. Contoh UI Detail	8
Gambar 3. Contoh Penerapan LazyRow/Carousel.....	9
Gambar 4. Screenshot Hasil Jawaban Soal Homepage Portrait (en) XML	51
Gambar 5. Screenshot Hasil Jawaban Soal DetailPage Portrait (en) XML	51
Gambar 6. Screenshot Hasil Jawaban Soal SettingPage Portrait (en) XML.....	52
Gambar 7. Screenshot Hasil Jawaban Soal Homepage Landscape (en) XML	52
Gambar 8. Screenshot Hasil Jawaban Soal DetailPage Landscape (en) XML	52
Gambar 9. Screenshot Hasil Jawaban Soal SettingPage Landscape (en) XML	53
Gambar 10. Screenshot Hasil Jawaban Soal Homepage Portrait (in) XML	53
Gambar 11. Screenshot Hasil Jawaban Soal DetailPage Portrait (in) XML.....	53
Gambar 12. Screenshot Hasil Jawaban Soal SettingPage Portrait (in) XML	54
Gambar 13. Screenshot Hasil Jawaban Soal Homepage Landscape (in) XML.....	54
Gambar 14. Screenshot Hasil Jawaban Soal DetailPage Landscape (in) XML.....	54
Gambar 15. Screenshot Hasil Jawaban Soal SettingPage Landscape (in) XML	55
Gambar 16. Screenshot Hasil Jawaban Soal Homepage Portrait (en) Compose	55
Gambar 17. Screenshot Hasil Jawaban Soal DetailPage Portrait (en) Compose.....	55
Gambar 18. Screenshot Hasil Jawaban Soal SettingPage Portrait (en) Compose	56
Gambar 19. Screenshot Hasil Jawaban Soal Homepage Landscape (en) Compose...	56
Gambar 20. Screenshot Hasil Jawaban Soal DetailPage Landscape (en) Compose...	56
Gambar 21. Screenshot Hasil Jawaban Soal SettingPage Landscape (en) Compose .	57
Gambar 22. Screenshot Hasil Jawaban Soal Homepage Portrait (in) Compose.....	57
Gambar 23. Screenshot Hasil Jawaban Soal Detail Page Portrait (in) Compose.....	57
Gambar 24. Screenshot Hasil Jawaban Soal Setting Page Portrait (in) Compose	58
Gambar 25. Screenshot Hasil Jawaban Soal Homepage Landscape (in) Compose....	58
Gambar 26. Screenshot Hasil Jawaban Soal Detail Page Landscape (in) Compose ..	58
Gambar 27. Screenshot Hasil Jawaban Soal Setting Page Landscape (in) Compose .	58

DAFTAR TABEL

Tabel 1. Source Code MainActivity.Kt XML.....	10
Tabel 2. Source Code ScrollableViewModel.Kt XML	10
Tabel 3. Source Code ScrollableUiState.Kt XML	11
Tabel 4. Source Code ScrollableData.Kt XML	11
Tabel 5. Source Code DataSource.Kt XML.....	12
Tabel 6. Source Code SettingsFragment.Kt XML	13
Tabel 7. Source Code DetailFragment.Kt XML	14
Tabel 8. Source Code HomeFragment.Kt XML	16
Tabel 9. Source Code CarouselAdapter.Kt XML	19
Tabel 10. Source Code CarouselHeaderAdapter.Kt XML.....	19
Tabel 11. Source Code ItemCardAdapter.Kt XML	20
Tabel 12. Source Code CarouselViewHolder.Kt XML	21
Tabel 13. Source Code ItemCardViewHolder.Kt XML	21
Tabel 14. Source Code navigation_graph.xml.....	22
Tabel 15. Source Code activity_main.xml.....	23
Tabel 16. Source Code fragment_detail.xml.....	23
Tabel 17. Source Code fragment_settings.xml	26
Tabel 18. Source Code fragment_home.xml.....	28
Tabel 19. Source Code menu_homel.xml.....	29
Tabel 20. Source Code home_carousel_header.xml	29
Tabel 21. Source Code home_carousel_card.xml.....	29
Tabel 22. Source Code home_item_card.xml	30
Tabel 23. Source Code MainActivity.Kt Compose	32
Tabel 24. Source Code ScrollableApp.Kt Compose	33
Tabel 25. Source Code ScrollableData.Kt Compose	35
Tabel 26. Source Code DataSource.Kt Compose	35
Tabel 27. Source Code ScrollableViewModel.Kt Compose.....	36
Tabel 28. Source Code ScrollableUiState.Kt Compose.....	37

Tabel 29. Source Code DetailScreen.Kt Compose	37
Tabel 30. Source Code SettingScreen.Kt Compose.....	40
Tabel 31. Source Code HomeScreen.Kt Compose	43

SOAL

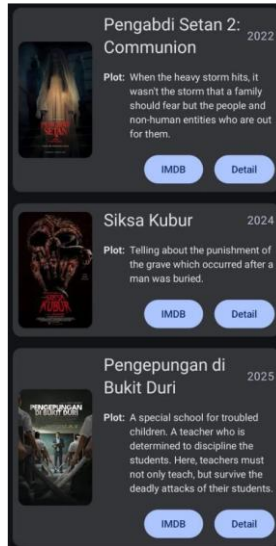
Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:

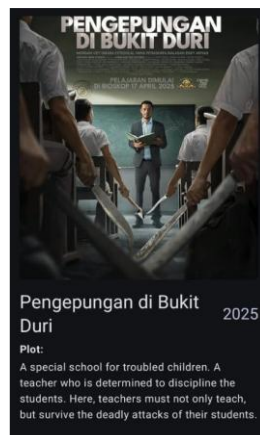
- Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:

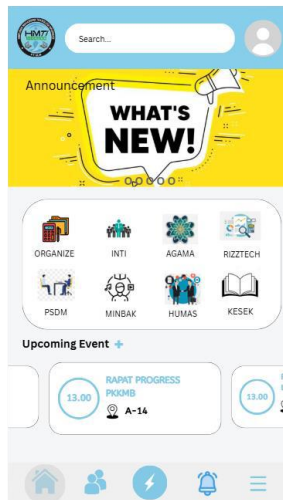


Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail



Gambar 3. Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5' kembali ke item awal '1')

A. Source Code

Tabel 1. Source Code MainActivity.Kt XML

1	package com.example.scrollablexml
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.WindowCompat
7	import
	com.example.scrollablexml.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	private lateinit var binding: ActivityMainBinding
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	enableEdgeToEdge()
14	WindowCompat.setDecorFitsSystemWindows(window,
	false)
15	binding =
	ActivityMainBinding.inflate(layoutInflater)
16	setContentView(binding.root)
17	}
18	}

Tabel 2. Source Code ScrollableViewModel.Kt XML

1	package com.example.scrollablexml.ui
2	
3	import androidx.appcompat.app.AppCompatActivity
4	import androidx.core.os.LocaleListCompat
5	import androidx.lifecycle.ViewModel
6	import com.example.scrollablexml.model.DataSource
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.asStateFlow
10	
11	class ScrollableViewModel : ViewModel() {
12	private val _uiState =
	MutableStateFlow(ScrollableUiState())
13	val uiState: StateFlow<ScrollableUiState> =
	_uiState.asStateFlow()
14	
15	init {
16	val items = DataSource().loadItems()
17	val currentLocale =
	AppCompatActivity.getApplicationLocales().get(0)?.language
	?: "en"

18	<code>_uiState.value = ScrollableUiState(</code>
19	<code>list = items,</code>
20	<code>currentItemIndex = 0,</code>
21	<code>selectedLocale = currentLocale</code>
22	<code>)</code>
23	<code>}</code>
24	
25	<code>fun updateCurrentItem(index: Int) {</code>
26	<code>_uiState.value = _uiState.value.copy(</code>
27	<code>currentItemIndex = index</code>
28	<code>)</code>
29	<code>}</code>
30	
31	<code>fun updateLocale(locale: String) {</code>
32	<code>_uiState.value = _uiState.value.copy(</code>
33	<code>selectedLocale = locale,</code>
34	<code>)</code>
35	<code>val appLocale =</code>
36	<code>LocaleListCompat.forLanguageTags(locale)</code>
37	<code>AppCompatDelegate.setApplicationLocales(appLocale)</code>
38	<code>}</code>

Tabel 3. Source Code ScrollableUiState.Kt XML

1	<code>package com.example.scrollablexml.ui</code>
2	
3	<code>import com.example.scrollablexml.model.ScrollableData</code>
4	
5	<code>data class ScrollableUiState(</code>
6	<code>val list: List<ScrollableData> = emptyList(),</code>
7	<code>val currentItemIndex: Int = 0,</code>
8	<code>val selectedLocale: String = "en"</code>
9	<code>)</code>

Tabel 4. Source Code ScrollableData.Kt XML

1	<code>package com.example.scrollablexml.model</code>
2	
3	<code>import androidx.annotation.DrawableRes</code>
4	<code>import androidx.annotation.StringRes</code>
5	
6	<code>data class ScrollableData(</code>
7	<code>val id: Int,</code>
8	<code>@StringRes val titleResourceId: Int,</code>
9	<code>@StringRes val subtitleResourceId: Int,</code>
10	<code>@StringRes val descriptionResourceId: Int,</code>
11	<code>@StringRes val detailResourceId: Int,</code>

12	@DrawableRes val imageResourceId: Int,
13	@DrawableRes val detailImageResourceId: Int,
14	val steamUrl: String
15	)

Tabel 5. Source Code DataSource.Kt XML

1	package com.example.scrollablexml.model
2	
3	import com.example.scrollablexml.R
4	
5	class DataSource {
6	fun loadItems(): List<ScrollableData> {
7	return listOf(
8	ScrollableData(1, R.string.item1,
	R.string.item1_sub, R.string.item1_desc,
	R.string.item1_detail, R.drawable.crosscode,
	R.drawable.crosscode_detail,
	"https://store.steampowered.com/app/368340/CrossCode/"),
9	ScrollableData(2, R.string.item2,
	R.string.item2_sub, R.string.item2_desc,
	R.string.item2_detail, R.drawable.hades2,
	R.drawable.hades2_detail,
	"https://store.steampowered.com/app/1145350/Hades_II/"),
10	ScrollableData(3, R.string.item3,
	R.string.item3_sub, R.string.item3_desc,
	R.string.item3_detail, R.drawable.nms,
	R.drawable.nms_detail,
	"https://store.steampowered.com/app/275850/No_Mans_Sky/"
	),
11	ScrollableData(4, R.string.item4,
	R.string.item4_sub, R.string.item4_desc,
	R.string.item4_detail, R.drawable.coe33,
	R.drawable.coe33_detail,
	"https://store.steampowered.com/app/1903340/Clair_Obscur
	_Expedition_33/"),
12	ScrollableData(5, R.string.item5,
	R.string.item5_sub, R.string.item5_desc,
	R.string.item5_detail, R.drawable.sekiro,
	R.drawable.sekiro_detail,
	"https://store.steampowered.com/app/814380/Sekiro_Shadow
	s_Die_Twice__GOTY_Edition/")
13	)
14	}
15	}

Tabel 6. Source Code SettingsFragment.Kt XML

1	package com.example.scrollablexml.ui.settings
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.Lifecycle
10	import androidx.lifecycle.lifecycleScope
11	import androidx.lifecycle.repeatOnLifecycle
12	import androidx.navigation.fragment.findNavController
13	import com.example.scrollablexml.R
14	import com.example.scrollablexml.databinding.FragmentSettingsBi nding
15	import com.example.scrollablexml.ui.ScrollableViewModel
16	import kotlinx.coroutines.launch
17	
18	
19	class SettingsFragment : Fragment() {
20	private var _binding: FragmentSettingsBinding? = null
21	private val binding get() = _binding!!
22	private val viewModel: ScrollableViewModel by activityViewModels()
23	
24	override fun onCreateView(25 inflater: LayoutInflater, container: ViewGroup?, 26 savedInstanceState: Bundle? 27): View { 28 _binding = FragmentSettingsBinding.inflate(inflater, container, false) 29 return binding.root 30 } 31
32	override fun onViewCreated(view: View, savedInstanceState: Bundle?) { 33 super.onViewCreated(view, savedInstanceState) 34 35 binding.toolbar.setNavigationOnClickListener { 36 findNavController().navigateUp() 37 } 38

39	viewLifecycleOwner.lifecycleScope.launch {
40	
	viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
41	viewModel.uiState.collect { uiState ->
42	val checkedLocaleIn = if
	(uiState.selectedLocale == "en")
43	R.id.rb_english else
	R.id.rb_indonesian
44	if
	(binding.rgLocale.checkedRadioButtonId !=
	checkedLocaleIn)
45	binding.rgLocale.check(checkedLocaleIn)
46	}
47	}
48	}
49	
50	binding.rgLocale.setOnCheckedChangeListener { _,
	checkedLocaleIn ->
51	val locale = if (checkedLocaleIn ==
	R.id.rb_english) "en" else "id"
52	viewModel.updateLocale(locale)
53	}
54	}
55	
56	override fun onDestroyView() {
57	super.onDestroyView()
58	_binding = null
59	}
60	}

Tabel 7. Source Code DetailFragment.Kt XML

1	package com.example.scrollablexml.ui.detail
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.Lifecycle
10	import androidx.lifecycle.lifecycleScope
11	import androidx.lifecycle.repeatOnLifecycle
12	import androidx.navigation.fragment.findNavController
13	import androidx.navigation.fragment.navArgs

```

14 import
   com.example.scrollablexml.databinding.FragmentDetailBind
   ing
15 import com.example.scrollablexml.ui.ScrollableViewModel
16 import kotlinx.coroutines.launch
17
18 class DetailFragment : Fragment() {
19     private var _binding: FragmentDetailBinding? = null
20     private val binding get() = _binding!!
21     private val viewModel: ScrollableViewModel by
activityViewModels()
22     private val args: DetailFragmentArgs by navArgs()
23
24     override fun onCreateView(
25         inflater: LayoutInflater,
26         container: ViewGroup?,
27         savedInstanceState: Bundle?
28     ): View {
29         _binding =
FragmentDetailBinding.inflate(inflater, container,
false)
30         return binding.root
31     }
32
33     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
34         super.onViewCreated(view, savedInstanceState)
35
36         binding.toolbar.setNavigationOnClickListener {
37             findNavController().navigateUp()
38         }
39
40         val index = args.itemIndex
41         viewModel.updateCurrentItem(index)
42
43         viewLifecycleOwner.lifecycleScope.launch {
44             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
RTED) {
45                 viewModel.uiState.collect { uiState ->
46                     val item =
uiState.list.getOrNull(index) ?: uiState.list.first()
47                     binding.apply {
48                         toolbar.setTitle(item.titleResourceId)

```

49	<code>ivDetail.setImageResource(item.imageResourceId)</code>
50	<code>tvTitle.setText(item.titleResourceId)</code>
51	<code>tvSubtitle.setText(item.subtitleResourceId)</code>
52	<code>tvDesc.setText(item.descriptionResourceId)</code>
53	<code>tvDetail.setText(item.detailResourceId)</code>
54	<code> }</code>
55	<code> }</code>
56	<code> }</code>
57	<code> }</code>
58	
59	
60	<code> }</code>
61	
62	<code> override fun onDestroyView() {</code>
63	<code> super.onDestroyView()</code>
64	<code> _binding = null</code>
65	<code> }</code>
66	<code>}</code>

Tabel 8. Source Code HomeFragment.Kt XML

1	<code>package com.example.scrollablexml.ui.home</code>
2	
3	<code>import android.content.Intent</code>
4	<code>import androidx.recyclerview.widget.ConcatAdapter</code>
5	<code>import android.os.Bundle</code>
6	<code>import androidx.fragment.app.Fragment</code>
7	<code>import android.view.LayoutInflater</code>
8	<code>import android.view.View</code>
9	<code>import android.view.ViewGroup</code>
10	<code>import androidx.fragment.app.activityViewModels</code>
11	<code>import androidx.lifecycle.Lifecycle</code>
12	<code>import androidx.lifecycle.LifecycleScope</code>
13	<code>import androidx.lifecycle.repeatOnLifecycle</code>
14	<code>import androidx.navigation.fragment.findNavController</code>
15	<code>import com.example.scrollablexml.R</code>
16	<code>import</code> <code>com.example.scrollablexml.databinding.FragmentHomeBinding</code>
17	<code>import com.example.scrollablexml.ui.ScrollableViewModel</code>


```

18 import
   com.example.scrollablexml.ui.home.adapter.CarouselHeader
   Adapter
19 import
   com.example.scrollablexml.ui.home.adapter.ItemCardAdapte
   r
20 import kotlinx.coroutines.launch
21 import androidx.core.net.toUri
22 import androidx.recyclerview.widget.LinearLayoutManager
23
24
25
26 class HomeFragment : Fragment() {
27     private var _binding: FragmentHomeBinding? = null
28     private val binding get() = _binding!!
29     private val viewModel: ScrollableViewModel by
   activityViewModels()
30
31     override fun onCreateView(
32         inflater: LayoutInflater, container: ViewGroup?,
33         savedInstanceState: Bundle?
34     ): View {
35         _binding = FragmentHomeBinding.inflate(inflater,
   container, false)
36         return binding.root
37     }
38
39     override fun onViewCreated(view: View,
   savedInstanceState: Bundle?) {
40         super.onViewCreated(view, savedInstanceState)
41
42         binding.toolbar.inflateMenu(R.menu.menu_home)
43         binding.toolbar.setOnMenuItemClickListener {
   item ->
44             when (item.itemId) {
45                 R.id.action_setting -> {
46
47                     findNavController().navigate(R.id.action_homeFragment_to
   _settingsFragment)
48                     true
49                 }
50                 else -> false
51             }
52         binding.toolbar.setNavigationOnClickListener {
53             requireActivity().finish()

```

```

54         }
55
56         viewLifecycleOwner.lifecycleScope.launch {
57
58             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
59             RTED) {
60                 viewModel.uiState.collect { uiState ->
61                     val carouselAdapter =
62                     CarouselHeaderAdapter(
63                         items = uiState.list,
64                         onDetailClick = { index ->
65                             val action =
66                             HomeFragmentDirections.actionHomeFragmentToDetailFragmen
67                             t(itemIndex = index)
68
69                             findNavController().navigate(action)
70                             }
71                             )
72                             val itemCardAdapter =
73                             ItemCardAdapter(
74                                 uiState.list,
75                                 onDetailClicked = { index ->
76                                     val action =
77                                     HomeFragmentDirections.actionHomeFragmentToDetailFragmen
78                                     t(itemIndex = index)
79
80                                     findNavController().navigate(action)
81                                     },
82                                     onSteamClicked = { url ->
83                                         startActivity(Intent(Intent.ACTION_VIEW, url.toUri()))
84                                         }
85                                         )
86                                         binding.rvMain.apply {
87                                             layoutManager =
88                                             LinearLayoutManager(requireContext())
89                                             adapter =
90                                             ConcatAdapter(carouselAdapter, itemCardAdapter)
91                                         }
92                                     }
93                                 }
94                             }
95                         }
96                     }
97                 }
98             }
99         }
100     }
101
102     override fun onDestroyView() {
103         super.onDestroyView()

```

87	<code>_binding = null</code>
88	<code>}</code>
89	<code>}</code>

Tabel 9. Source Code CarouselAdapter.Kt XML

1	<code>package com.example.scrollablexml.ui.home.adapter</code>
2	
3	<code>import android.view.LayoutInflater</code>
4	<code>import android.view.ViewGroup</code>
5	<code>import androidx.recyclerview.widget.RecyclerView</code>
6	<code>import</code>
	<code>com.example.scrollablexml.databinding.HomeCarouselCardBi</code>
	<code>nding</code>
7	<code>import com.example.scrollablexml.model.ScrollableData</code>
8	<code>import</code>
	<code>com.example.scrollablexml.ui.home.viewholder.CarouselVie</code>
	<code>wHolder</code>
9	
10	<code>class CarouselAdapter(</code>
11	<code> private val items: List<ScrollableData>,</code>
12	<code> private val onDetailClicked: (Int) -> Unit</code>
13	<code>): RecyclerView.Adapter<CarouselViewHolder>() {</code>
14	
15	<code> override fun onCreateViewHolder(parent: ViewGroup,</code>
	<code>viewType: Int): CarouselViewHolder {</code>
16	<code> val binding = HomeCarouselCardBinding.inflate(</code>
17	<code> LayoutInflater.from(parent.context), parent,</code>
	<code>false</code>
18	<code>)</code>
19	<code> return CarouselViewHolder(binding)</code>
20	<code> }</code>
21	
22	<code> override fun onBindViewHolder(holder:</code>
	<code>CarouselViewHolder, position: Int) {</code>
23	<code> holder.bind(items[position], position,</code>
	<code>onDetailClicked)</code>
24	<code> }</code>
25	
26	<code> override fun getItemCount() = items.size</code>
27	<code>}</code>

Tabel 10. Source Code CarouselHeaderAdapter.Kt XML

1	<code>package com.example.scrollablexml.ui.home.adapter</code>
2	
3	<code>import android.view.ViewGroup</code>
4	<code>import androidx.recyclerview.widget.RecyclerView</code>

5	import com.example.scrollablexml.model.ScrollableData
6	import
	com.example.scrollablexml.ui.home.viewholder.CarouselHeaderViewHolder
7	
8	class CarouselHeaderAdapter(
9	private val items: List<ScrollableData>,
10	private val onDetailClick: (Int) -> Unit
11	): RecyclerView.Adapter<CarouselHeaderViewHolder>() {
12	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int)
13	= CarouselHeaderViewHolder.from(parent)
14	
15	override fun onBindViewHolder(holder:
	CarouselHeaderViewHolder, position: Int)
16	= holder.bind(items, onDetailClick)
17	
18	override fun getItemCount() = 1
19	}

Tabel 11. Source Code ItemCardAdapter.Kt XML

1	package com.example.scrollablexml.ui.home.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.scrollablexml.databinding.
	HomeItemCardBinding
7	import com.example.scrollablexml.model.ScrollableData
8	import
	com.example.scrollablexml.ui.home.viewholder.ItemCardViewHolder
9	
10	class ItemCardAdapter(
11	private val items: List<ScrollableData>,
12	private val onDetailClicked: (Int) -> Unit,
13	private val onSteamClicked: (String) -> Unit
14	): RecyclerView.Adapter<ItemCardViewHolder>() {
15	
16	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): ItemCardViewHolder {
17	val binding = HomeItemCardBinding.inflate(
18	LayoutInflater.from(parent.context), parent,
	false
19	)
20	return ItemCardViewHolder(binding)

21	}
22	
23	override fun onBindViewHolder(holder:
	ItemCardViewHolder, position: Int) {
24	holder.bind(items[position], position,
	onDetailClicked, onSteamClicked)
25	}
26	
27	override fun getItemCount() = items.size
28	}

Tabel 12. Source Code CarouselViewHolder.Kt XML

1	package com.example.scrollablexml.ui.home.viewholder
2	
3	import androidx.recyclerview.widget.RecyclerView
4	import com.example.scrollablexml.databinding.
	HomeCarouselCardBinding
5	import com.example.scrollablexml.model.ScrollableData
6	
7	class CarouselViewHolder(
8	private val binding: HomeCarouselCardBinding
9	): RecyclerView.ViewHolder(binding.root) {
10	fun bind(item: ScrollableData, index: Int,
	onDetailClicked: (Int) -> Unit) {
11	
	binding.ivCarousel.setImageResource(item.detailImageReso
	urceId)
12	binding.root.setOnClickListener {
	onDetailClicked(index) }
13	}
14	}

Tabel 13. Source Code ItemCardViewHolder.Kt XML

1	package com.example.scrollablexml.ui.home.viewholder
2	
3	import androidx.recyclerview.widget.RecyclerView
4	import
	com.example.scrollablexml.databinding.HomeItemCardBindin
	g
5	import com.example.scrollablexml.model.ScrollableData
6	
7	class ItemCardViewHolder(
8	private val binding: HomeItemCardBinding
9	): RecyclerView.ViewHolder(binding.root) {
10	
11	fun bind(

12	item: ScrollableData,
13	index: Int,
14	onDetailClicked: (Int) -> Unit,
15	onSteamClicked: (String) -> Unit
16	) {
17	binding.apply {
18	ivItem.setImageResource(item.imageResourceId)
19	tvTitle.setText(item.titleResourceId)
20	tvSubtitle.setText(item.subtitleResourceId)
21	tvDesc.setText(item.descriptionResourceId)
22	btnDetail.setOnClickListener {
23	onDetailClicked(index) }
24	btnSteam.setOnClickListener {
25	onSteamClicked(item.steamUrl) }
26	}

Tabel 14. Source Code navigation_graph.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:id="@+id/navigation_graph"
5	app:startDestination="@+id/homeFragment">
6	
7	<fragment
8	android:id="@+id/homeFragment"
9	
	android:name="com.example.scrollablexml.ui.home.HomeFragment"
10	android:label="@string/home_label">
11	<action
12	
	android:id="@+id/action_homeFragment_to_settingsFragment"
13	app:destination="@id/settingsFragment"/>
14	<action
15	
	android:id="@+id/action_homeFragment_to_detailFragment"
16	app:destination="@id/detailFragment"/>
17	</fragment>
18	
19	<fragment

20	android:id="@+id/detailFragment"
21	android:name="com.example.scrollablexml.ui.detail.Detail Fragment"
22	android:label="@string/detail_label">
23	<argument
24	android:name="itemIndex"
25	app:argType="integer"
26	android:defaultValue="0"/>
27	</fragment>
28	
29	<fragment
30	android:id="@+id/settingsFragment"
31	android:name="com.example.scrollablexml.ui.settings.Sett ingsFragment"
32	android:label="@string/setting_label"/>
33	
34	</navigation>

Tabel 15. Source Code activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView xmlns:android="http://schemas.android.com/apk/res/androi d"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:name="androidx.navigation.fragment.NavHostFragme nt"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:defaultNavHost="true"
10	app:navGraph="@navigation/navigation_graph"
11	tools:layout="@layout/fragment_home" />

Tabel 16. Source Code fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true">

```

7
8     <com.google.android.material.appbar.AppBarLayout
9         android:layout_width="match_parent"
10        android:layout_height="wrap_content">
11
12
13    <com.google.android.material.appbar.MaterialToolbar
14        android:id="@+id/toolbar"
15        android:layout_width="match_parent"
16        android:layout_height="wrap_content"
17        android:minHeight="?attr/actionBarSize"
18        app:titleCentered="true"
19        app:navigationIcon="@drawable/ic_arrow_back"
20        android:layout_marginTop="8dp"/>
21
22    </com.google.android.material.appbar.AppBarLayout>
23
24    <androidx.core.widget.NestedScrollView
25        android:layout_width="match_parent"
26        android:layout_height="match_parent"
27
28        app:layout_behavior="@string/appbar_scrolling_view_
29        behavior">
30
31        <LinearLayout
32            android:layout_width="match_parent"
33            android:layout_height="wrap_content"
34            android:padding="16dp"
35            android:orientation="vertical">
36
37            <ImageView
38                android:id="@+id/iv_detail"
39                android:layout_width="350dp"
40                android:layout_height="450dp"
41
42                android:contentDescription="@string/head_title"
43                android:layout_gravity="center"/>
44
45            <LinearLayout
46                android:layout_width="match_parent"
47                android:layout_height="wrap_content"
48                android:orientation="vertical"
49                android:padding="16dp">
50
51                <TextView
52                    android:id="@+id/tv_title"

```


49	android:layout_width="wrap_content"
50	android:layout_height="wrap_content"
51	android:textAppearance="?attr/ textAppearanceHeadlineMedium"/>
52	
53	<TextView
54	android:id="@+id/tv_subtitle"
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:textAppearance="?attr/ textAppearanceHeadlineSmall"/>
58	
59	<com.google.android.material.card.MaterialCardView
60	android:layout_width="match_parent"
61	android:layout_height="wrap_content"
62	android:layout_marginTop="8dp"
63	app:cardElevation="4dp"
64	app:strokeWidth="1dp"
65	app:strokeColor="?attr/colorOutline">
66	
67	<LinearLayout
68	android:layout_width="match_parent"
69	android:layout_height="wrap_content"
70	android:orientation="vertical"
71	android:padding="8dp">
72	
73	<TextView
74	android:id="@+id/tv_desc"
75	android:layout_width="wrap_content"
76	android:layout_height="wrap_content"
77	android:textAppearance="?attr/ textAppearanceTitleLarge"/>
78	
79	<TextView

80	android:id="@+id/tv_detail"
81	android:layout_width="wrap_content"
82	android:layout_height="wrap_content"
83	android:textAppearance="?attr/textAppearanceBodyMedium"
84	android:justificationMode="inter_word"/>
85	
86	</LinearLayout>
87	
88	</com.google.android.material.card.MaterialCardView>
89	
90	</LinearLayout>
91	</LinearLayout>
92	</androidx.core.widget.NestedScrollView>
93	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 17. Source Code fragment settings.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true">
7	
8	<com.google.android.material.appbar.AppBarLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<com.google.android.material.appbar.MaterialToolbar
13	android:id="@+id/toolbar"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:minHeight="?attr/actionBarSize"
17	app:title="@string/setting_button"
18	app:titleCentered="true"
19	app:navigationIcon="@drawable/ic_arrow_back"
20	android:layout_marginTop="8dp"/>
21	

```

22     </com.google.android.material.appbar.AppBarLayout>
23
24     <androidx.core.widget.NestedScrollView
25         android:layout_width="match_parent"
26         android:layout_height="match_parent"
27         app:layout_behavior="@string/
appbar_scrolling_view_behavior">
28
29         <LinearLayout
30             android:layout_width="match_parent"
31             android:layout_height="wrap_content"
32             android:orientation="vertical"
33             android:padding="16dp">
34
35             <TextView
36                 android:layout_width="wrap_content"
37                 android:layout_height="wrap_content"
38                 android:text="@string/setting_header"
39                 android:textAppearance="?attr/
textAppearanceHeadlineSmall"
40                 android:layout_marginVertical="8dp"/>
41
42             <RadioGroup
43                 android:id="@+id/rg_locale"
44                 android:layout_width="wrap_content"
45                 android:layout_height="wrap_content"
46                 android:orientation="vertical">
47
48                 <RadioButton
49                     android:id="@+id/rb_english"
50                     android:layout_width="wrap_content"
51                     android:layout_height="wrap_content"
52                     android:text="@string/button_english"/>
53
54                 <RadioButton
55                     android:id="@+id/rb_indonesian"
56                     android:layout_width="wrap_content"
57                     android:layout_height="wrap_content"
58                     android:text="@string/button_indonesia"/>
59
60             </RadioGroup>
61
62         </LinearLayout>
63

```

64	</androidx.core.widget.NestedScrollView>
65	
66	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 18. Source Code fragment home.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/ android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:fitsSystemWindows="true"
6	xmlns:app="http://schemas.android.com/apk/res-auto">
7	
8	<com.google.android.material.appbar.AppBarLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content">
11	
12	<com.google.android.material.appbar.MaterialToolbar
13	android:id="@+id/toolbar"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:minHeight="?attr/actionBarSize"
17	app:titleCentered="true"
18	
19	app:navigationIcon="@drawable/ic_exit_to_app"
20	android:layout_marginTop="8dp">
21	<TextView
22	android:id="@+id/tv_toolbar_title"
23	android:layout_width="wrap_content"
24	android:layout_height="wrap_content"
25	android:layout_gravity="center_vertical"
26	android:text="@string/head_title"
27	android:maxLines="2"
28	android:gravity="start"
29	android:textAppearance="?attr/ textAppearanceHeadlineSmall" />
30	
31	</com.google.android.material.appbar.MaterialToolbar>
32	
33	</com.google.android.material.appbar.AppBarLayout>
34	
35	<androidx.recyclerview.widget.RecyclerView

36	android:id="@+id/rv_main"
37	android:layout_width="match_parent"
38	android:layout_height="match_parent"
39	android:padding="8dp"
40	android:clipToPadding="false"
41	app:layout_behavior="@string/ appbar_scrolling_view_behavior"/>
42	
43	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 19. Source Code menu homel.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto">
4	
5	<item
6	android:id="@+id/action_setting"
7	android:title="@string/setting_button"
8	android:icon="@drawable/ic_settings"
9	app:showAsAction="ifRoom"/>
10	
11	</menu>

Tabel 20. Source Code home carousel header.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.recyclerview.widget.RecyclerView xmlns:android="http://schemas.android.com/apk/res/ android"
3	android:id="@+id/rv_carousel"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:orientation="horizontal"/>

Tabel 21. Source Code home carousel card.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/ android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:layout_margin="8dp"
7	app:cardElevation="4dp"
8	app:strokeWidth="1dp"
9	app:strokeColor="?attr/colorOutline">

10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="horizontal"
15	android:padding="16dp"
16	android:gravity="center">
17	
18	<com.google.android.material.imageview.
19	ShapeableImageView
20	android:id="@+id/iv_carousel"
21	android:layout_width="400dp"
22	android:layout_height="200dp"
23	android:scaleType="centerCrop"
24	app:shapeAppearanceOverlay="@style/ShapeAppearance.
25	Material3.Corner.Medium"/>
26	</LinearLayout>
27	</com.google.android.material.card.MaterialCardView>

Tabel 22. Source Code home item card.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
	xmlns:android="http://schemas.android.com/apk/res/
	android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:layout_margin="8dp"
7	app:cardElevation="4dp"
8	app:strokeWidth="1dp"
9	app:strokeColor="?attr/colorOutline">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="horizontal"
15	android:padding="16dp">
16	
17	<com.google.android.material.imageview.
18	ShapeableImageView
19	android:id="@+id/iv_item"
20	android:layout_width="100dp"
	android:layout_height="140dp"

```

21         android:scaleType="fitXY"
22
23     app:shapeAppearanceOverlay="@style/ShapeAppearance.
24     Material3.Corner.Medium"/>
25
26     <LinearLayout
27         android:layout_width="0dp"
28         android:layout_height="wrap_content"
29         android:layout_weight="1"
30         android:layout_marginStart="16dp"
31         android:orientation="vertical">
32
33         <TextView
34             android:id="@+id/tv_title"
35             android:layout_width="wrap_content"
36             android:layout_height="wrap_content"
37             android:textAppearance="?attr/
38             textAppearanceHeadlineSmall"/>
39
40         <TextView
41             android:id="@+id/tv_subtitle"
42             android:layout_width="wrap_content"
43             android:layout_height="wrap_content"
44             android:textAppearance="?attr/textAppearanceBodyLarge"/>
45
46         <TextView
47             android:id="@+id/tv_desc"
48             android:layout_width="wrap_content"
49             android:layout_height="wrap_content"
50             android:textAppearance="?attr/
51             textAppearanceBodyMedium"/>
52
53         <LinearLayout
54             android:layout_width="match_parent"
55             android:layout_height="wrap_content"
56             android:layout_marginTop="24dp"
57             android:orientation="horizontal"
58             android:gravity="end">
59
60         <com.google.android.material.button.MaterialButton
61             android:id="@+id/btn_steam"
62             android:layout_width="wrap_content"
63             android:layout_height="wrap_content"
64             android:text="@string/steam_button"

```

61	android:maxLines="1"
62	
63	android:textAppearance="?attr/textAppearanceBodySmall"
64	android:paddingHorizontal="16dp"/>
65	<Space
66	android:layout_width="8dp"
67	
68	android:layout_height="wrap_content"/>
69	
70	<com.google.android.material.button.MaterialButton
71	android:id="@+id/btn_detail"
72	android:layout_width="wrap_content"
73	android:layout_height="wrap_content"
74	android:text="@string/detail_button"
75	android:maxLines="1"
76	android:textAppearance="?attr/textAppearanceBodySmall"/>
77	</LinearLayout>
78	</LinearLayout>
79	
80	</com.google.android.material.card.MaterialCardView>

Tabel 23. Source Code MainActivity.Kt Compose

1	package com.example.scrollablemodul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import
8	com.example.scrollablemodul3.ui.theme.ScrollableModul3Theme
9	class MainActivity : AppCompatActivity() {
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	enableEdgeToEdge()
13	setContent {
14	ScrollableModul3Theme {
15	ScrollableApp()
16	}
17	}
18	}

19	}
----	---

Tabel 24. Source Code ScrollableApp.Kt Compose

1	package com.example.scrollablemodul3
2	
3	import android.app.Activity
4	import android.content.Intent
5	import androidx.compose.runtime.Composable
6	import androidx.compose.runtime.LaunchedEffect
7	import androidx.compose.runtime.collectAsState
8	import androidx.compose.runtime.getValue
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.platform.LocalContext
11	import androidx.core.net.toUri
12	import androidx.lifecycle.viewmodel.compose.viewModel
13	import androidx.navigation.NavHostController
14	import androidx.navigation.compose.NavHost
15	import androidx.navigation.compose.composable
16	import androidx.navigation.compose.rememberNavController
17	import
	com.example.scrollablemodul3.ui.screen.DetailScreen
18	import com.example.scrollablemodul3.ui.screen.HomeScreen
19	import
	com.example.scrollablemodul3.ui.ScrollableViewModel
20	import
	com.example.scrollablemodul3.ui.screen.SettingScreen
21	
22	enum class ScrollableScreen { Home, Details, Settings }
23	
24	@Composable
25	fun ScrollableApp(
26	viewModel: ScrollableViewModel = viewModel(),
27	navController: NavHostController =
	rememberNavController()
28	) {
29	val uiState by viewModel.uiState.collectAsState()
30	val context = LocalContext.current
31	
32	NavHost(
33	navController = navController,
34	startDestination = ScrollableScreen.Home.name,
35	) {
36	composable(route = ScrollableScreen.Home.name) {
37	val activity = context as Activity
38	HomeScreen(
39	uiState = uiState,

```

40         onDetailClick = { index ->
41
42             navController.navigate("${ScrollableScreen.Details.name}
43             /$index")
44
45             },
46             onIntentClick = { url ->
47
48                 context.startActivity(Intent(Intent.ACTION_VIEW,
49                 url.toUri()))
50
51                 },
52                 onSettingsClick = {
53                 navController.navigate(ScrollableScreen.Settings.name)
54                 },
55                 onExit = { activity.finish() }
56             )
57         }
58         composable(route =
59         "${ScrollableScreen.Details.name}/{itemId}") {
60         backStackEntry ->
61             val index =
62             backStackEntry.arguments?.getString("itemId")?.toIntOrNull() ?: 0
63
64             val item = uiState.list.getOrNull(index) ?:
65             uiState.list.first()
66             LaunchedEffect(index) {
67                 viewModel.updateCurrentItem(index)
68             }
69             DetailScreen(
70                 item = item,
71                 modifier = Modifier,
72                 onBackClick = {
73                 navController.navigateUp() }
74             )
75         }
76         composable(route =
77         ScrollableScreen.Settings.name) {
78             SettingScreen(
79                 modifier = Modifier,
80                 onBackClick = {
81                 navController.navigateUp() },
82                 onLocaleChange = { locale ->
83                 viewModel.updateLocale(locale) },
84                 selectedLocale = uiState.selectedLocale
85             )
86         }
87     }
88 }

```

71	}
----	---

Tabel 25. Source Code ScrollableData.Kt Compose

1	package com.example.scrollablemodul3.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class ScrollableData(
7	val id: Int,
8	@StringRes val titleResourceId: Int,
9	@StringRes val subtitleResourceId: Int,
10	@StringRes val descriptionResourceId: Int,
11	@StringRes val detailResourceId: Int,
12	@DrawableRes val imageResourceId: Int,
13	@DrawableRes val detailImageResourceId: Int,
14	val steamUrl: String
15	)

Tabel 26. Source Code DataSource.Kt Compose

1	package com.example.scrollablemodul3.model
2	
3	import com.example.scrollablemodul3.R
4	
5	class DataSource {
6	fun loadItems(): List<ScrollableData> {
7	return listOf(
8	ScrollableData(1, R.string.item1,
	R.string.item1_sub, R.string.item1_desc,
	R.string.item1_detail, R.drawable.crosscode,
	R.drawable.crosscode_detail,
	"https://store.steampowered.com/app/368340/CrossCode/"),
9	ScrollableData(2, R.string.item2,
	R.string.item2_sub, R.string.item2_desc,
	R.string.item2_detail, R.drawable.hades2,
	R.drawable.hades2_detail,
	"https://store.steampowered.com/app/1145350/Hades_II/"),
10	ScrollableData(3, R.string.item3,
	R.string.item3_sub, R.string.item3_desc,
	R.string.item3_detail, R.drawable.nms,
	R.drawable.nms_detail,
	"https://store.steampowered.com/app/275850/No_Mans_Sky/"
	),
11	ScrollableData(4, R.string.item4,
	R.string.item4_sub, R.string.item4_desc,
	R.string.item4_detail, R.drawable.coe33,

12	R.drawable.coe33_detail, "https://store.steampowered.com/app/1903340/Clair_Obscur _Expedition_33/"), ScrollableData(5, R.string.item5, R.string.item5_sub, R.string.item5_desc, R.string.item5_detail, R.drawable.sekiro, R.drawable.sekiro_detail, "https://store.steampowered.com/app/814380/Sekiro_Shadow s_Die_Twice__GOTY_Edition/")
13	)
14	}
15	}

Tabel 27. Source Code ScrollableViewModel.Kt Compose

1	package com.example.scrollablemodul3.ui
2	
3	import androidx.appcompat.app.AppCompatActivityDelegate
4	import androidx.core.os.LocaleListCompat
5	import androidx.lifecycle.ViewModel
6	import com.example.scrollablemodul3.model.DataSource
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.asStateFlow
10	
11	class ScrollableViewModel : ViewModel() {
12	private val _uiState =
	MutableStateFlow(ScrollableUiState())
13	val uiState: StateFlow<ScrollableUiState> =
	_uiState.asStateFlow()
14	
15	init {
16	val items = DataSource().loadItems()
17	val currentLocale =
	AppCompatActivity.getApplicationLocales().get(0)?.language ? : "en"
18	_uiState.value = ScrollableUiState(
19	list = items,
20	currentItemIndex = 0,
21	selectedLocale = currentLocale
22	)
23	}
24	
25	fun updateCurrentItem(index: Int) {
26	_uiState.value = _uiState.value.copy(
27	currentItemIndex = index
28	)

29	}
30	
31	fun updateLocale(locale: String) {
32	_uiState.value = _uiState.value.copy(
33	selectedLocale = locale,
34	)
35	val appLocale =
36	LocaleListCompat.forLanguageTags(locale)
37	AppCompatActivity.setApplicationLocales(appLocale)
38	}

Tabel 28. Source Code ScrollableUiState.Kt Compose

1	package com.example.scrollablemodul3.ui
2	
3	import com.example.scrollablemodul3.model.ScrollableData
4	
5	data class ScrollableUiState(
6	val list: List<ScrollableData> = emptyList(),
7	val currentItemIndex: Int = 0,
8	val selectedLocale: String = "en"
9	)

Tabel 29. Source Code DetailScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.fillMaxWidth
7	import androidx.compose.foundation.layout.height
8	import androidx.compose.foundation.layout.padding
9	import androidx.compose.foundation.rememberScrollState
10	import
	androidx.compose.foundation.shape.RoundedCornerShape
11	import androidx.compose.foundation.verticalScroll
12	import androidx.compose.material.icons.Icons
13	import
	androidx.compose.material.icons.automirrored.filled.
	ArrowBack
14	import androidx.compose.material3.Card
15	import androidx.compose.material3.CardDefaults
16	import androidx.compose.material3.CenterAlignedTopAppBar
17	import
	androidx.compose.material3.ExperimentalMaterial3Api

```

18 import androidx.compose.material3.Icon
19 import androidx.compose.material3.IconButton
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Scaffold
22 import androidx.compose.material3.Text
23 import androidx.compose.runtime.Composable
24 import androidx.compose.ui.Modifier
25 import androidx.compose.ui.res.painterResource
26 import androidx.compose.ui.res.stringResource
27 import androidx.compose.ui.text.style.TextAlign
28 import androidx.compose.ui.tooling.preview.Preview
29 import androidx.compose.ui.unit.dp
30 import com.example.scrollablemodul3.R
31 import com.example.scrollablemodul3.model.ScrollableData
32
33 @OptIn(ExperimentalMaterial3Api::class)
34 @Composable
35 fun DetailScreen(
36     item: ScrollableData,
37     onBackClick: () -> Unit,
38     modifier: Modifier = Modifier
39 ){
40     Scaffold(
41         topBar = {
42             CenterAlignedTopAppBar(
43                 title = {
44                     Text(text = stringResource(id =
45 item.titleResourceId))
46                 },
47                 navigationIcon = {
48                     IconButton(onClick = onBackClick) {
49                         Icon(
50                             imageVector =
51 Icons.AutoMirrored.Filled.ArrowBack,
52                             contentDescription =
53 stringResource(R.string.back_button)
54                         )
55                     }
56                 }
57             )
58         }
59     ) { innerPadding ->
60         Column(
61             modifier = modifier
62                 .padding(innerPadding)
63                 .verticalScroll(rememberScrollState())

```

```

61         ) {
62             Image(
63                 painter = painterResource(id =
item.imageResourceId),
64                 contentDescription = stringResource(id =
item.titleResourceId),
65                 modifier = Modifier
66                     .fillMaxWidth()
67                     .height(450.dp)
68             )
69
70             Column(modifier = Modifier.padding(16.dp)) {
71                 Text(
72                     text = stringResource(id =
item.titleResourceId),
73                     style =
MaterialTheme.typography.headlineMedium
74                 )
75                 Text(
76                     text = stringResource(id =
item.subtitleResourceId),
77                     style =
MaterialTheme.typography.headlineSmall
78                 )
79                 Card(
80                     modifier = Modifier
81                         .fillMaxWidth()
82                         .padding(top = 8.dp),
83                     elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
84                     shape = RoundedCornerShape(8.dp),
85                     border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline)
86                 ) {
87                     Column(modifier =
Modifier.padding(8.dp)) {
88                         Text(
89                             text = stringResource(id =
item.descriptionResourceId),
90                             style =
MaterialTheme.typography.titleLarge
91                         )
92                         Text(
93                             text = stringResource(id =
item.detailResourceId),

```

94	style =
95	MaterialTheme.typography.bodyMedium,
	textAlign =
	TextAlign.Justify
96	)
97	}
98	}
99	}
100	}
101	}
102	}
103	
104	@Preview
105	@Composable
106	fun DetailScreenPreview() {
107	DetailScreen(
108	item = ScrollableData(
109	id = 3,
110	titleResourceId = R.string.item3,
111	subtitleResourceId = R.string.item3_sub,
112	descriptionResourceId = R.string.item3_desc,
113	detailResourceId = R.string.item3_detail,
114	imageResourceId = R.drawable.nms,
115	detailImageResourceId =
	R.drawable.nms_detail,
116	steamUrl =
	"https://store.steampowered.com/app/1091500/Sekiro_Shadow_Die_Twice/"
117	),
118	modifier = Modifier,
119	onBackClick = {}
120	)
121	}

Tabel 30. Source Code SettingScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.layout.Column
4	import androidx.compose.foundation.layout.Row
5	import androidx.compose.foundation.layout.fillMaxWidth
6	import androidx.compose.foundation.layout.padding
7	import androidx.compose.foundation.rememberScrollState
8	import androidx.compose.foundation.verticalScroll
9	import androidx.compose.material.icons.Icons
10	import androidx.compose.material.icons.automirrored.filled.ArrowBack


```

11 import androidx.compose.material3.CenterAlignedTopAppBar
12 import
13     androidx.compose.material3.ExperimentalMaterial3Api
14 import androidx.compose.material3.Icon
15 import androidx.compose.material3.IconButton
16 import androidx.compose.material3.MaterialTheme
17 import androidx.compose.material3.RadioButton
18 import androidx.compose.material3.Scaffold
19 import androidx.compose.material3.Text
20 import androidx.compose.runtime.Composable
21 import androidx.compose.ui.Alignment
22 import androidx.compose.ui.Modifier
23 import androidx.compose.ui.res.stringResource
24 import androidx.compose.ui.tooling.preview.Preview
25 import androidx.compose.ui.unit.dp
26 import com.example.scrollablemodul3.R
27
28 @OptIn(ExperimentalMaterial3Api::class)
29 @Composable
30 fun SettingScreen(
31     modifier: Modifier = Modifier,
32     onBackClick: () -> Unit,
33     onLocaleChange: (String) -> Unit,
34     selectedLocale: String
35 ){
36     Scaffold(
37         topBar = {
38             CenterAlignedTopAppBar(
39                 title = {
40                     Text(text =
41 stringResource(R.string.setting_button))
42                 },
43                 navigationIcon = {
44                     IconButton(onClick = onBackClick) {
45                         Icon(
46                             imageVector =
47 Icons.AutoMirrored.Filled.ArrowBack,
48                             contentDescription =
49 stringResource(R.string.back_button)
50                         )
51                     }
52                 }
53             )
54         }
55     ) {
56         innerPadding ->
57         Column(

```

```

53         modifier = modifier
54         .padding(innerPadding)
55         .padding(16.dp)
56         .fillMaxWidth()
57         .verticalScroll(rememberScrollState())
58     ) {
59         Text(
60             text =
61             stringResource(R.string.setting_header),
62             style =
63             MaterialTheme.typography.headlineSmall,
64             modifier = Modifier.padding(vertical =
65             8.dp)
66         )
67         Row(
68             verticalAlignment =
69             Alignment.CenterVertically,
70             modifier = Modifier.fillMaxWidth()
71         ) {
72             RadioButton(
73                 selected = selectedLocale == "en",
74                 onClick = { onLocaleChange("en") }
75             )
76             Text(text =
77             stringResource(R.string.english_button))
78         }
79         Row(
80             verticalAlignment =
81             Alignment.CenterVertically,
82             modifier = Modifier.fillMaxWidth()
83         ) {
84             RadioButton(
85                 selected = selectedLocale == "in",
86                 onClick = { onLocaleChange("in") }
87             )
88             Text(text =
89             stringResource(R.string.indonesian_button))
90         }
91     }
92 }
93
94 @Preview
95 @Composable
96 fun SettingScreenLayout() {

```

92	SettingScreen(
93	modifier = Modifier,
94	onBackClick = {},
95	onLocaleChange = {},
96	selectedLocale = "en"
97	)
98	}

Tabel 31. Source Code HomeScreen.Kt Compose

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.gestures.snapping. rememberSnapFlingBehavior
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.PaddingValues
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.layout.width
15	import androidx.compose.foundation.lazy.LazyColumn
16	import androidx.compose.foundation.lazy.LazyRow
17	import androidx.compose.foundation.lazy.itemsIndexed
18	import androidx.compose.foundation.lazy.rememberLazyListState
19	import androidx.compose.foundation.shape.RoundedCornerShape
20	import androidx.compose.material.icons.Icons
21	import androidx.compose.material.icons.automirrored.filled. ExitToApp
22	import androidx.compose.material.icons.filled.Settings
23	import androidx.compose.material3.Button
24	import androidx.compose.material3.Card
25	import androidx.compose.material3.CardDefaults
26	import androidx.compose.material3.ExperimentalMaterial3Api
27	import androidx.compose.material3.Icon
28	import androidx.compose.material3.IconButton
29	import androidx.compose.material3.MaterialTheme
30	import androidx.compose.material3.Scaffold
31	import androidx.compose.material3.Text

```

32 import androidx.compose.material3.TopAppBar
33 import androidx.compose.runtime.Composable
34 import androidx.compose.ui.Modifier
35 import androidx.compose.ui.draw.clip
36 import androidx.compose.ui.layout.ContentScale
37 import androidx.compose.ui.res.painterResource
38 import androidx.compose.ui.res.stringResource
39 import androidx.compose.ui.tooling.preview.Preview
40 import androidx.compose.ui.unit.dp
41 import com.example.scrollablemodul3.R
42 import com.example.scrollablemodul3.model.ScrollableData
43 import com.example.scrollablemodul3.ui.ScrollableUiState
44
45 @OptIn(ExperimentalMaterial3Api::class)
46 @Composable
47 fun HomeAppBar(
48     onSettingClick: () -> Unit,
49     onExit: () -> Unit
50 ) {
51     TopAppBar(
52         title = {
53             Text(stringResource(R.string.head_title))
54         },
55         navigationIcon = {
56             IconButton(onClick = onExit) {
57                 Icon(
58                     imageVector =
59 Icons.AutoMirrored.Filled.ExitToApp,
60                     contentDescription =
61 stringResource(R.string.back_button)
62                 )
63             }
64         },
65         actions = {
66             IconButton(onClick = onSettingClick) {
67                 Icon(
68                     imageVector =
69 Icons.Default.Settings,
70                     contentDescription =
71 stringResource(R.string.setting_button)
72                 )
73             }
74         }
75     )
76 }
77 @Composable

```

```

74 fun HomeScreen(
75     uiState: ScrollableUiState,
76     onDetailClick: (Int) -> Unit,
77     onIntentClick: (String) -> Unit,
78     onSettingsClick: () -> Unit,
79     onExit: () -> Unit,
80     modifier: Modifier = Modifier
81 ){
82     Scaffold(
83         topBar = {
84             HomeAppBar(
85                 onSettingClick = onSettingsClick,
86                 onExit = onExit
87             )
88         }
89     ) { innerPadding ->
90         LazyColumn(modifier = modifier
91             .padding(innerPadding)
92             .padding(start = 8.dp)
93         ) {
94             item {
95                 ItemCarousel(
96                     items = uiState.list,
97                     onDetailClick = { index ->
onDetailClick(index) }
98                 )
99             }
100             itemsIndexed(uiState.list) { index, item ->
101                 ItemCard(
102                     item = item,
103                     onDetailClick = {
onDetailClick(index) },
104                     onIntentClick = {
onIntentClick(item.steamUrl) }
105                 )
106             }
107         }
108     }
109 }
110
111 @Composable
112 fun ItemCard(
113     item: ScrollableData,
114     onDetailClick: () -> Unit,
115     onIntentClick: () -> Unit,
116     modifier: Modifier = Modifier

```

```

117 ) {
118     Card(
119         modifier = modifier
120             .padding(8.dp)
121             .fillMaxWidth(),
122         shape = MaterialTheme.shapes.medium,
123         elevation =
124         CardDefaults.cardElevation(defaultElevation = 4.dp),
125         border = BorderStroke(1.dp,
126         MaterialTheme.colorScheme.outline)
127     ) {
128         Row(modifier = Modifier.padding(16.dp)) {
129             Image(
130                 painter = painterResource(id =
131                 item.imageResourceId),
132                 contentDescription = stringResource(id =
133                 item.titleResourceId),
134                 contentScale = ContentScale.FillBounds,
135                 modifier = Modifier
136                     .width(100.dp)
137                     .height(140.dp)
138                     .clip(RoundedCornerShape(16.dp))
139             )
140             Column(
141                 modifier = Modifier
142                     .padding(start = 16.dp)
143                     .weight(1f)
144             ) {
145                 Text(
146                     text = stringResource(id =
147                     item.titleResourceId),
148                     style =
149                     MaterialTheme.typography.headlineSmall
150                 )
151                 Text(
152                     text = stringResource(id =
153                     item.subtitleResourceId),
154                     style =
155                     MaterialTheme.typography.bodyLarge
156                 )
157                 Text(
158                     text = stringResource(id =
159                     item.descriptionResourceId),
160                     style =
161                     MaterialTheme.typography.bodyMedium
162                 )
163             }
164         }
165     }
166 }

```

```

153
154         Row(
155             modifier = Modifier
156                 .padding(top = 24.dp)
157                 .fillMaxWidth(),
158             horizontalArrangement =
Arrangement.End,
159         ) {
160             Button(
161                 onClick = onIntentClick,
162                 contentPadding =
PaddingValues(horizontal = 16.dp)
163             ) {
164                 Text(
165                     text =
stringResource(R.string.steam_button),
166                     style =
MaterialTheme.typography.bodySmall
167                 )
168             }
169             Spacer(modifier =
Modifier.padding(8.dp))
170             Button(
171                 onClick = onDetailClick,
172             ) {
173                 Text(
174                     text =
stringResource(R.string.detail_button),
175                     style =
MaterialTheme.typography.bodySmall
176                 )
177             }
178         }
179     }
180 }
181 }
182 }
183
184 @Composable
185 fun CarouselCard(
186     item: ScrollableData,
187     onDetailClick: () -> Unit,
188     modifier: Modifier = Modifier
189 ){
190     Card(
191         onClick = onDetailClick,

```

```

192         modifier = modifier.padding(8.dp),
193         shape = MaterialTheme.shapes.medium,
194         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
195         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline),
196         colors = CardDefaults.cardColors(
197             containerColor =
MaterialTheme.colorScheme.surfaceContainer
198         )
199     ) {
200         Row(
201             modifier = Modifier
202                 .padding(16.dp)
203                 .fillMaxWidth(),
204             horizontalArrangement =
Arrangement.SpaceEvenly
205         ) {
206             Image(
207                 painter = painterResource(id =
item.detailImageResourceId),
208                 contentDescription = stringResource(id =
item.titleResourceId),
209                 contentScale = ContentScale.Crop,
210                 modifier = Modifier
211                     .width(400.dp)
212                     .height(200.dp)
213                     .clip(RoundedCornerShape(16.dp))
214             )
215         }
216     }
217 }
218
219
220 @Composable
221 fun ItemCarousel(
222     items: List<ScrollableData>,
223     onDetailClick: (Int) -> Unit,
224 ) {
225     val listState = rememberLazyListState()
226     LazyRow(
227         state = listState,
228         flingBehavior =
rememberSnapFlingBehavior(listState)
229     ) {
230         itemsIndexed(items) { index, item ->

```



```

231         CarouselCard(
232             item = item,
233             onDetailClick = { onDetailClick(index)
        },
234             modifier = Modifier.fillParentMaxWidth()
235         )
236     }
237 }
238 }
239
240 @Preview
241 @Composable
242 fun HomeScreenPreview() {
243     HomeScreen(
244         uiState = ScrollableUiState(
245             list = listOf(
246                 ScrollableData(
247                     id = 1,
248                     titleResourceId = R.string.item1,
249                     subtitleResourceId =
R.string.item1_sub,
250                     descriptionResourceId =
R.string.item1_desc,
251                     detailResourceId =
R.string.item1_detail,
252                     imageResourceId =
R.drawable.crosscode,
253                     detailImageResourceId =
R.drawable.crosscode_detail,
254                     steamUrl =
"https://store.steampowered.com/app/368340/CrossCode/"
                ),
255                 ScrollableData(
256                     id = 2,
257                     titleResourceId = R.string.item2,
258                     subtitleResourceId =
R.string.item2_sub,
259                     descriptionResourceId =
R.string.item2_desc,
260                     detailResourceId =
R.string.item2_detail,
261                     imageResourceId = R.drawable.hades2,
262                     detailImageResourceId =
R.drawable.hades2_detail,
263                     steamUrl =
"https://store.steampowered.com/app/368340/CrossCode/"
                )
            )
        )
    }
}

```

```

265         ),
266         ScrollableData(
267             id = 3,
268             titleResourceId = R.string.item3,
269             subtitleResourceId =
R.string.item3_sub,
270             descriptionResourceId =
R.string.item3_desc,
271             detailResourceId =
R.string.item3_detail,
272             imageResourceId = R.drawable.nms,
273             detailImageResourceId =
R.drawable.nms_detail,
274             steamUrl =
"https://store.steampowered.com/app/368340/CrossCode/"
275         ),
276         ScrollableData(
277             id = 4,
278             titleResourceId = R.string.item4,
279             subtitleResourceId =
R.string.item4_sub,
280             descriptionResourceId =
R.string.item4_desc,
281             detailResourceId =
R.string.item4_detail,
282             imageResourceId = R.drawable.coe33,
283             detailImageResourceId =
R.drawable.coe33_detail,
284             steamUrl =
"https://store.steampowered.com/app/368340/CrossCode/"
285         ),
286         ScrollableData(
287             id = 5,
288             titleResourceId = R.string.item5,
289             subtitleResourceId =
R.string.item5_sub,
290             descriptionResourceId =
R.string.item5_desc,
291             detailResourceId =
R.string.item5_detail,
292             imageResourceId = R.drawable.sekiro,
293             detailImageResourceId =
R.drawable.sekiro_detail,
294             steamUrl =
"https://store.steampowered.com/app/368340/CrossCode/"
295         ),

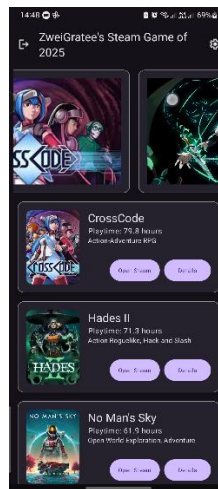
```

```

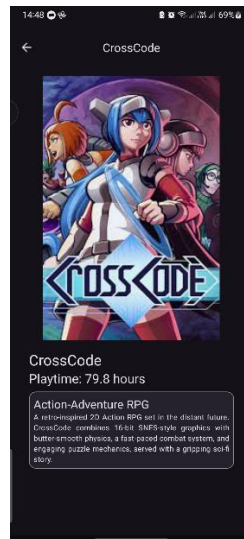
296         )
297     ),
298
299     onDetailClick = {},
300     onIntentClick = {},
301     onSettingsClick = {},
302     onExit = {}
303 )
304 }

```

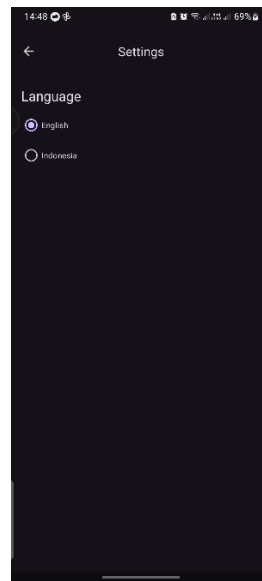
B. Output Program



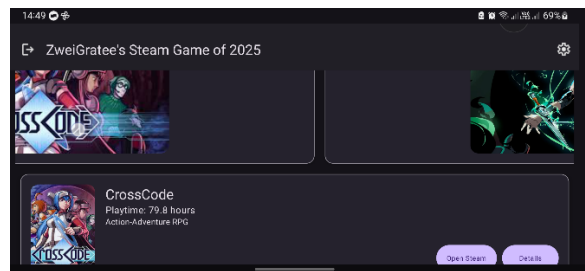
Gambar 4. Screenshot Hasil Jawaban Soal Homepage Portrait (en) XML



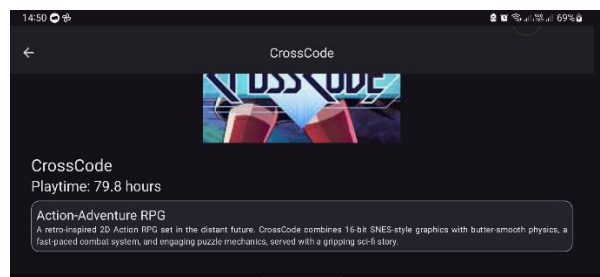
Gambar 5. Screenshot Hasil Jawaban Soal DetailPage Portrait (en) XML



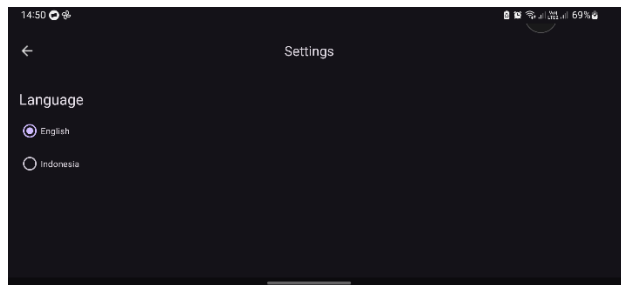
Gambar 6. Screenshot Hasil Jawaban Soal SettingPage Portrait (en) XML



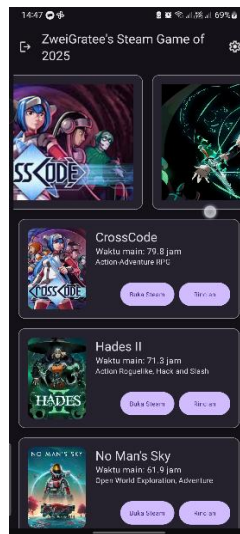
Gambar 7. Screenshot Hasil Jawaban Soal Homepage Landscape (en) XML



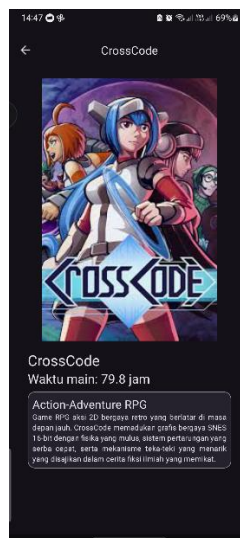
Gambar 8. Screenshot Hasil Jawaban Soal DetailPage Landscape (en) XML



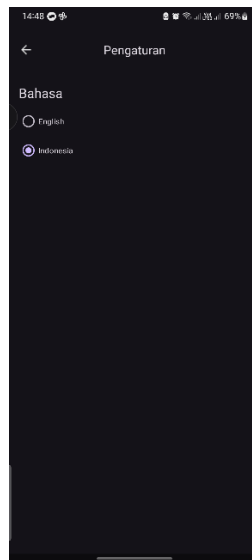
Gambar 9. Screenshot Hasil Jawaban Soal SettingPage Landscape (en) XML



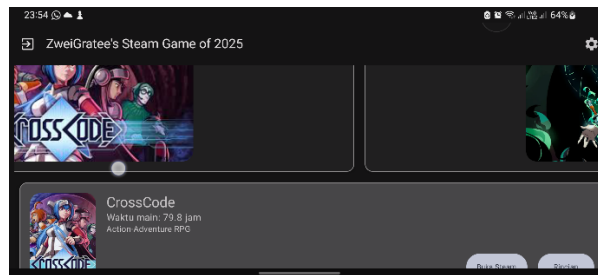
Gambar 10. Screenshot Hasil Jawaban Soal Homepage Portrait (in) XML



Gambar 11. Screenshot Hasil Jawaban Soal DetailPage Portrait (in) XML



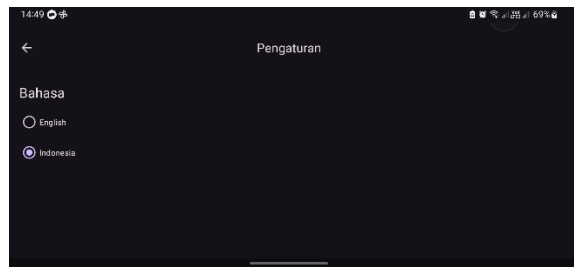
Gambar 12. Screenshot Hasil Jawaban Soal SettingPage Portrait (in) XML



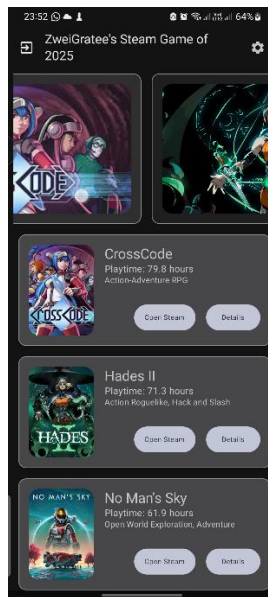
Gambar 13. Screenshot Hasil Jawaban Soal Homepage Landscape (in) XML



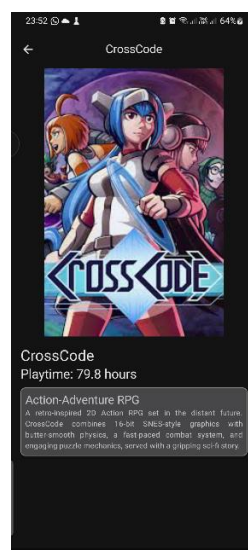
Gambar 14. Screenshot Hasil Jawaban Soal DetailPage Landscape (in) XML



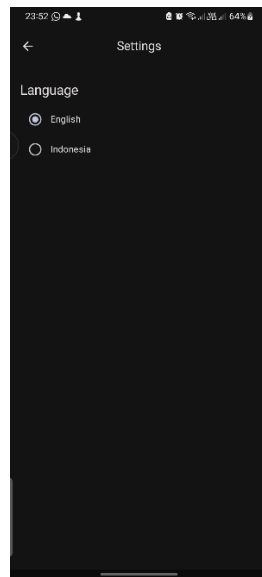
Gambar 15. Screenshot Hasil Jawaban Soal SettingPage Landscape (in) XML



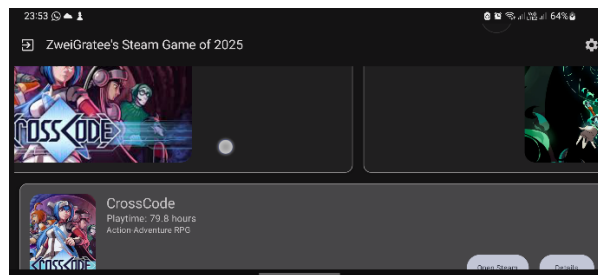
Gambar 16. Screenshot Hasil Jawaban Soal Homepage Portrait (en) Compose



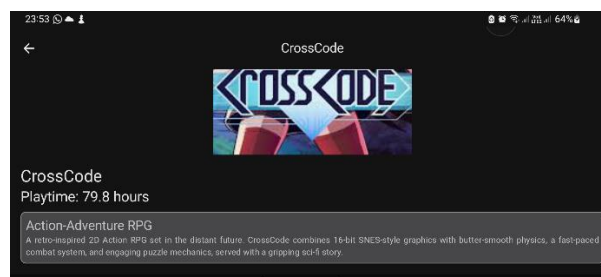
Gambar 17. Screenshot Hasil Jawaban Soal DetailPage Portrait (en) Compose



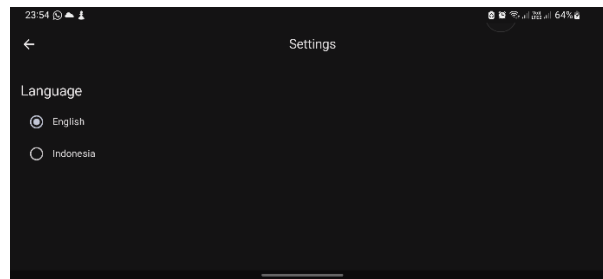
Gambar 18. Screenshot Hasil Jawaban Soal SettingPage Portrait (en) Compose



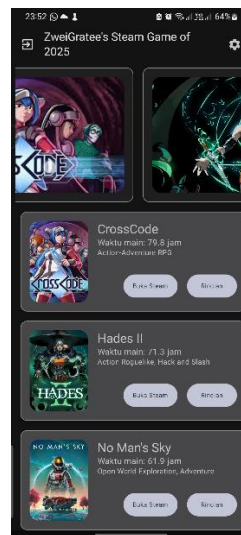
Gambar 19. Screenshot Hasil Jawaban Soal Homepage Landscape (en) Compose



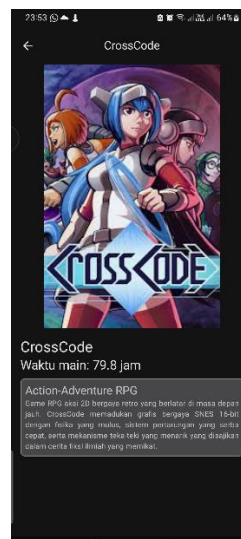
Gambar 20. Screenshot Hasil Jawaban Soal DetailPage Landscape (en) Compose



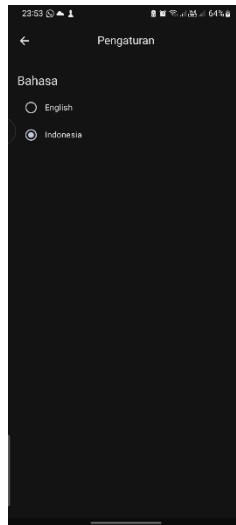
Gambar 21. Screenshot Hasil Jawaban Soal SettingPage Landscape (en) Compose



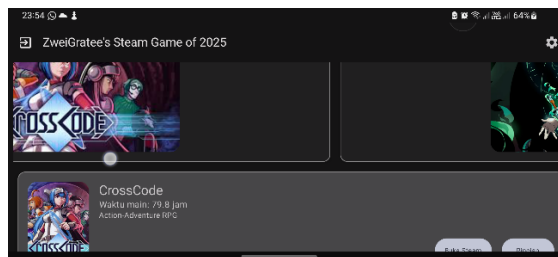
Gambar 22. Screenshot Hasil Jawaban Soal Homepage Portrait (in) Compose



Gambar 23. Screenshot Hasil Jawaban Soal Detail Page Portrait (in) Compose



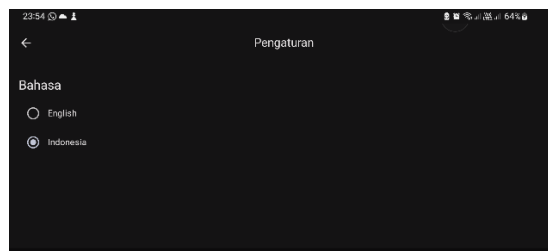
Gambar 24. Screenshot Hasil Jawaban Soal Setting Page Portrait (in) Compose



Gambar 25. Screenshot Hasil Jawaban Soal Homepage Landscape (in) Compose



Gambar 26. Screenshot Hasil Jawaban Soal Detail Page Landscape (in) Compose



Gambar 27. Screenshot Hasil Jawaban Soal Setting Page Landscape (in) Compose

C. Pembahasan

ScrollableData.Kt & DataSource.Kt

ScrollableData.Kt merupakan data class yang berfungsi untuk menyimpan jenis-jenis data yang akan dimuat ke dalam aplikasi dengan data berupa id bertipe data Integer, data title, subtitle, description, dan detail yang merupakan String Resource, data image dan detail image yang merupakan Drawable Resource, dan data URL steam bertipe data String.

Data-data yang didefinisikan pada ScrollableData.Kt kemudian dimuat dengan method “loadItems()” yang akan memuat data-data hardcoded ke dalam aplikasi. Data yang dimuat disimpan dalam variable bertipe data List dan terdapat 5 data hardcoded yang akan dimuat ke dalam aplikasi yang nantinya akan dikelola oleh ViewModel.

ScrollableUiState.Kt & ScrollableViewModel.Kt

ScrollableUiState.Kt merupakan data class StateFlow yang menyimpan dan menginisialisasikan data berupa state atau kondisi UI yang terdapat pada aplikasi. State yang disimpan pada data class tersebut adalah “list” yang menyimpan data terkait list item yang akan ditampilkan pada Home Page, “currentItemIndex” yang menyimpan indeks list item yang berfungsi sebagai helper untuk navigasi ke Detail Page, dan “selectedLocale” yang menyimpan konfigurasi bahasa Aplikasi.

ScrollableViewModel.Kt merupakan class yang berfungsi untuk mengelola logika UI aplikasi dengan menerapkan arsitektur Model-View-ViewModel yang memisahkan data, logika bisnis, dan logika UI. Pada inisialisasi ViewModel, data-data aplikasi akan diinisialisasi, indeks list item diinisialisasi, dan locale aplikasi akan diambil kemudian data tersebut disimpan pada ScrollabelUiState.

Method “updateCurrentItem()” berfungsi untuk memperbarui indeks list item yang dipilih pada ScrollableViewState.Kt yang nantinya akan digunakan untuk melakukan navigasi ke Detail Page sesuai dengan indeks item yang dipilih. Method “updateLocale()” berfungsi untuk mengatur locale atau konfigurasi bahasa aplikasi dengan memperbarui locale pada ScrollableViewState.Kt kemudian mengubah konfigurasi bahasa sesuai parameter.

MainActivity.Kt, activity_main.xml & navigation_graph.xml

MainActivity.Kt berfungsi sebagai Activity/UI untuk Aplikasi android yang akan memuat UI yang diambil dari layout file XML. Melalui ViewBinding, terdapat inisialisasi variabel “binding” yang berfungsi sebagai view binder ke layout activity_main.xml yang akan dihubungkan saat Activity menjalankan method “onCreate()” saat aplikasi pertama kali dibuka. Styling aplikasi akan extend ke navigation bar dan status bar, tetapi komponen UI tidak akan membelakangi kedua bar tersebut.

activity_main.xml berfungsi sebagai layar utama yang akan dimuat saat method “onCreate()” dijalankan oleh Activity. Layar utama ini hanya berfungsi sebagai navigator yang akan digunakan oleh komponen navigasi pada file navigation_graph.xml yang memuat tiga Fragment yang telah dibuat dan dihubungkan pada navigation_graph.xml dengan layout default berupa Fragment Homepage.

navigation_graph.xml berfungsi untuk mengatur navigasi antar layar atau UI dengan memuat Fragment suatu page melalui method “onCreateView” dan di-“hancurkan” melalui method “onDestroyView” pada masing-masing fragment. Pada komponen navigasi, terdapat tiga Fragment yang dihubungkan, yaitu fragment_home.xml, fragment_detail.xml, dan fragment_settings.xml. Pada Fragment Home, terdapat dua action yang menghubungkan fragment tersebut ke Fragment Detail dan Fragment Setting, dimana action tersebut digunakan pada file HomeFragment.Kt.

fragment_settings.xml & SettingsFragment.Kt XML

fragment_settings.xml merupakan layout yang digunakan oleh Fragment Setting Page yang terdiri dari “CoordinatorLayout” yang merupakan layout yang berfungsi untuk memuat Topbar dan layar UI dibawahnya. Topbar didefinisikan dengan ViewGroup “AppBarLayout” yang memuat layout Topbar dan “MaterialToolbar” dengan padding atas 8dp yang memuat tampilan Topbar seperti ikon navigasi dengan desain Material 3 dan teks title.

Tampilan layar pada Fragment Settings diatur dengan “NestedScrollView” yang merupakan “ScrollView” modern yang diterapkan di dalam “CoordinatorLayout” sebagai layar scrollable. Di dalamnya, terdapat “LinearLayout” dengan orientasi vertikal dan padding keseluruhan 16dp yang memuat View “TextView” yang memuat teks dengan gaya Material 3 Headline dan margin vertikal 8dp. Kemudian, terdapat ViewGroup “RadioGroup” yang berfungsi untuk memuat kumpulan View “RadioButton” dengan orientasi vertikal.

SettingsFragment.Kt merupakan Fragment yang berupa “mini activity” yang lebih ringan dibandingkan penggunaan activity langsung sebagai Settings Page yang menampilkan UI berupa fragment_settings.xml dengan lifecycle tersendiri. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_setting.xml. Setelah Fragment dibangun atau pada siklus “onViewCreated()”, Fragment menghubungkan listener ke tombol-tombol untuk navigasi back dan konfigurasi locale yang terdapat pada UI. Jika Fragment ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

fragment_detail & DetailFragment.Kt XML

fragment_detail.xml merupakan layout yang digunakan oleh Fragment Detail Page yang terdiri dari “CoordinatorLayout” berfungsi untuk memuat Topbar dengan “AppBarLayout” dan layar UI dibawahnya dengan “NestedScrollView”. Topbar didefinisikan dengan ViewGroup “AppBarLayout” yang memuat layout Topbar dan View “MaterialToolbar” dengan padding atas 8dp yang memuat tampilan Topbar seperti ikon navigasi dengan desain Material 3 dan teks title.

Tampilan layar pada Fragment Detail diatur dengan “NestedScrollView” yang berfungsi sebagai layar scrollable. Di dalamnya, terdapat “LinearLayout” vertikal dengan padding keseluruhan 16dp yang memuat “ImageView” dengan tinggi 350dp dan lebar 450dp dan orientasi gambar tengah. Di bawahnya, terdapat “LinearLayout” vertikal dengan padding keseluruhan 16dp yang memuat “TextView” title dan subtitle dengan gaya Material 3 Headline.

Dibawah “TextView title dan subtitle, terdapat ViewGroup “MaterialCardView” yang memuat layout Card atau kartu yang menampilkan suatu tempat dengan margin atas 8dp, elevasi kartu 4dp, ketebalan outline 1dp dengan warna Material 3 Outline. Di dalamnya terdapat “LinearLayout” vertikal dengan padding keseluruhan 8dp yang memuat “TextView” deskripsi yang memakai gaya Material 3 Title dan detail yang memakai gaya Material 3 Body.

DetailFragment.Kt merupakan Fragment yang berfungsi sebagai Detail Page yang menampilkan UI dengan lifecycle tersendiri. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_detail.xml. Setelah Fragment dibangun atau pada siklus “onViewCreated()”, Fragment menghubungkan data-data yang dimuat ke dalam komponen teks dan gambar serta navigasi back. Jika Fragment ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

fragment_home.xml, menu_home.xml, & HomeFragment.Kt XML

fragment_home.xml merupakan layout yang digunakan oleh Fragment HomePage yang terdiri dari “CoordinatorLayout” yang memuat Topbar dengan ViewGroup “AppBarLayout” yang terdiri dari “MaterialToolbar” dengan padding atas 8dp yang menampilkan ikon navigasi exit serta menu_home.xml yang menampilkan ikon navigasi setting. Di dalam “MaterialToolbar”, terdapat view “TextView” sebagai title dengan orientasi tengah vertikal dan teks align left dan 2 baris teks dengan gaya Material 3 Headline. Dibawahnya terdapat ViewGroup “RecyclerView” yang menampilkan list data banyak secara dinamis efisien dengan mendaur ulang elemen View.

HomeFragment.Kt merupakan Fragment yang berfungsi sebagai Homepage. Pada siklus “onCreateView()”, Fragment akan membangun UI dari file fragment_home.xml. Pada siklus “onViewCreated()”, Fragment menghubungkan listener ke tombol-tombol navigasi pada item list dan Carousel, intent ke Steam Page, dan exit yang terdapat pada UI serta menghubungkan Adapter dan data ke UI. Jika Fragment ditutup atau pada siklus “onViewDestroy()”, Fragment akan melepas layout XML yang telah dibangun dengan memindahkan pointer ke nilai null.

home_carousel_header.xml,

CarouselHeaderAdapter.Kt, & CarouselHeaderViewHoder.Kt XML

home_carousel_header.xml merupakan layout yang akan digunakan oleh Carousel yang terdapat pada Homepage yang ditampilkan melalui ViewGroup “RecyclerView”. Data-data yang ingin ditampilkan akan dikoneksikan ke RecyclerView melalui Adapter dimana data tersebut akan dikoneksikan ke masing-masing View terlebih dahulu melalui ViewHolder.

CarouselHeaderViewHolder.Kt merupakan class yang menghubungkan data-data dan behaviour yang akan dimuat ke dalam ViewGroup “RecyclerView” Carousel dengan posisi yang diatur secara horizontal dan bagaimana Carousel dihubungkan ke file XML. CarouselHeaderAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data yang dihubungkan pada ViewHolder serta behavior jika Carousel ditekan.

home_carousel_card.xml,

CarouselAdapter.Kt, & CarouselViewHolder.Kt XML

home_carousel_card.xml merupakan layout komponen-komponen UI yang akan membangun Carousel yang telah dibuat yang terdiri dari “MaterialCardView” dengan margin keseluruhan 8dp, elevasi 4dp, ketebalan outline 1dp dan warna dari Material 3 Outline. Di dalamnya, terdapat “LinearLayout” horizontal dengan padding 16dp dan orientasi child View tengah. Di dalamnya, terdapat View “ShapeableImageView” yang merupakan komponen gambar dengan gaya yang dapat diatur. Tinggi “ShapeableImageView” adalah 400dp dan lebarnya 200dp, isi gambar memotong dirinya jika memenuhi View, dan gaya View menggunakan Material 3 Corner untuk Rounded Corner.

CarouselHeaderViewHolder.Kt merupakan class yang menghubungkan data-data dan behaviour yang akan dimuat ke dalam ViewGroup “MaterialCardView” dan View “ShapeableImageView” CarouselAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data yang telah dihubungkan pada ViewHolder serta behavior jika Carousel ditekan.

home_item_card.xml,

ItemCardAdapter.Kt, & ItemCardViewHolder.Kt XML

home_item_card.xml merupakan layout komponen-komponen UI yang akan membangun item list yang terdiri dari “MaterialCardView” dengan margin keseluruhan 8dp, elevasi 4dp, ketebalan outline 1dp dan warna dari Material 3 Outline. Di dalamnya, terdapat “LinearLayout” horizontal dengan padding 16dp dan orientasi child View tengah. Di dalamnya, terdapat View “ShapeableImageView” dengan tinggi 100dp dan lebar 140dp, isi gambar memenuhi View, dan gaya View menggunakan Material 3 Corner untuk Rounded Corner.

Di dalamnya, terdapat “LinearLayout” vertikal dengan margin awal 16dp yang memuat View “TextView” title dengan gaya teks Material 3 Headline, “TextView” subtitle dan description dengan gaya teks Material 3 Body. Di bawahnya, terdapat “LinearLayout” horizontal dengan margin atas 24dp dan orientasi komponen view di akhir. Di dalamnya, terdapat View “MaterialButton” yang memuat tombol dengan gaya Material 3 dengan tombol pertama memiliki padding horizontal 16dp berfungsi sebagai explicit intent ke Steam Page dan tombol kedua berfungsi untuk navigasi ke Detail Page dengan gaya teks Material 3 Body dan dipisah dengan View “Space” berjarak 8dp.

ItemCardViewHolder.Kt merupakan class yang menghubungkan data-data yang akan dimuat ke dalam masing-masing view yang terdapat pada home_item_card.xml. ItemCardAdapter.Kt merupakan class Adapter yang melakukan koneksi pada layout UI dengan data-data yang telah dihubungkan pada ViewHolder serta behavior jika tombol ditekan.

MainActivity.Kt & ScrollableApp.Kt Compose

MainActivity.Kt berfungsi sebagai Activity utama dari Aplikasi Android dengan konfigurasi edge to edge, tema aplikasi menyesuaikan tema sistem Android, dan layout yang diambil dari method composable “ScrollableApp()” di file ScrollableApp.Kt. Pada file ScrollableApp.Kt, terdapat Class yang menyimpan tipe data enum dengan nama “ScrollableScreen” yang berisikan tiga konstanta yang dipakai untuk navigasi, yaitu “Home”, “Details”, dan “Settings”.

Method composable “ScrollableApp()” berfungsi untuk menampilkan layout utama Aplikasi pada Activity serta navigasi aplikasi per layar screen. Navigasi dilakukan dengan “NavHost” dan “NavController” dengan layar default Homepage dan tiga rute navigasi yang dikoneksikan melalui method navigator “composable()”. Pada rute Homepage, terdapat inisialisasi composable “HomeScreen()” yang menginisialisasikan UI State dari ScrollableUiState, logika tombol detail yang bernavigasi ke screen Detail Page sesuai nilai indeks, logika tombol intent yang menjalankan explicit intent ke Steam Page, logika tombol setting yang bernavigasi ke screen Setting Page, dan logika tombol exit yang menutup aplikasi.

Pada rute Detail Page, nilai indeks item akan diperbarui secara asynchronous, kemudian menginisialisasikan composable “DetailScreen()” dengan parameter “item” yang menentukan item mana yang dimuat dari data, “modifier” sebagai modifier layout, dan “onBackClick” sebagai logika tombol back. Pada rute Setting Page, terdapat inisialisasi composable “SettingScreen()” dengan parameter “modifier” sebagai modifier layout, “onBackClick” sebagai logika tombol back, “onLocaleChange” sebagai logika konfigurasi bahasa yang memanggil method pada ViewModel, dan “selectedLocale” yang menyimpan State Locale ke ScrollableUiState.

SettingScreen.Kt Compose

Method Composable “SettingScreen()” berfungsi untuk memuat layout UI pada Setting Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title atau judul dan ikon navigasi yang diambil dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawah Topbar terdiri dari “Column” yang memuat kolom dengan padding yang menyesuaikan komponen “Scaffold” dan jarak keseluruhan 16dp dan menyimpan UI serta dapat melakukan scroll vertikal dan menyimpan posisi scroll.

Di dalam komponen “Column”, terdapat komponen “Text” yang memuat teks pengaturan/setting dengan gaya dari Material 3 Headline dan padding vertikal 8dp. Kemudian, terdapat tiga komponen yang berfungsi untuk memuat masing-masing UI Radio Button, yaitu komponen “Row” yang memuat baris dengan orientasi tengah vertikal yang didalamnya terdapat komponen “RadioButton” yang terpilih jika nilai “selectedLocale” sesuai dengan Locale Sistem dan panggilan method untuk mengubah Locale, serta komponen “Text” yang mengasih keterangan pada tiap komponen “RadioButton”. Method preview “SettingScreenLayout()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

DetailScreen.Kt Compose

Method Composable “DetailScreen()” berfungsi untuk memuat layout UI pada Detail Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title dengan komponen “Text” dan ikon navigasi dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawahnya terdiri dari komponen kolom “Column” scrollable dengan padding menyesuaikan kerangka.

Di dalam komponen “Column”, terdapat komponen gambar “Image” yang mengambil sumber gambar dari data dengan lebar memenuhi parent dan tinggi 450dp. Di bawahnya, terdapat komponen “Column” dengan padding keseluruhan 16dp yang memuat komponen “Text” title dan subtitle dengan gaya teks Material 3 Headline. Di bawahnya, terdapat komponen “Card” yang memuat suatu tempat dengan padding atas 8dp, elevasi 4dp, bentuk pojok melengkung sebesar 8dp, outline setebal 1dp dengan gaya Material 3 outline. Di dalam komponen “Card”, terdapat komponen “Column” dengan padding 8dp yang memuat komponen “Text” description dengan gaya Material 3 Title dan “Text” detail dengan gaya Material 3 Body. Method preview “DetailScreenPreview()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

HomeScreen.Kt Compose

Method Composable “HomeAppBar()” berfungsi untuk memuat layout Topbar UI yang terdiri dari Topbar biasa yang terdiri dari sub komponen title, ikon navigasi exit, dan ikon navigasi actions yang digunakan untuk berpindah ke Setting Page. Method Composable “HomeScreen()” berfungsi untuk memuat layout UI Homepage menggunakan komponen “Scaffold” sebagai kerangka yang terdiri dari Topbar yang diambil dari method composable “HomeAppBar()” dengan parameter “OnSettingClick” untuk berpindah ke setting dan parameter “OnExit” untuk keluar dari aplikasi.

Isi dari “HomeScreen()” adalah komponen “LazyColumn” dengan padding menyesuaikan kerangka “Scaffold” dan padding awal 8dp yang memuat UI scrollable yang hanya terlihat pada layar. “LazyColumn” digunakan untuk memuat data-data yang dimuat pada Carousel yang didefinisikan pada composable “ItemCarousel()” serta logika tombol detail. Selain itu, juga terdapat pemuatan data-data berbasis indeks ke item list yang didefinisikan pada composable “ItemCard()” dengan logika tombol detail serta tombol dengan explicit intent.

Composable “ItemCard()” berfungsi untuk memuat UI item list yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, dan border atau outline setebal 1dp dengan warna Material 3 outline. Di dalamnya, terdapat komponen “Row” dengan padding keseluruhan 16dp yang berisikan komponen gambar “Image” yang mengambil sumbernya dari data pada DataSource.Kt, isi gambar memenuhi View, dan modifier tinggi 140dp dan lebar 100dp serta ukuran pojok melengkung sebesar 16dp.

Di sebelah komponen “Image”, terdapat komponen “Column” dengan padding awal 16dp dan komponen mengambil setengah dari space parent View. Di dalamnya, terdapat komponen “Text” yang memuat teks title dengan gaya material 3 Headline, teks subtitle dan description dengan gaya material 3 body. Di bawahnya, terdapat komponen “Row” dengan padding atas 24dp dan orientasi child View berada di akhir. Di dalamnya, terdapat komponen tombol “Button” dengan tombol pertama memiliki padding horizontal 16dp dan kedua teks pada tombol memiliki gaya Material 3 body. Kedua tombol dipisah dengan komponen “Spacer” dengan jarak 8dp.

Composable “CarouselCard()” berfungsi untuk memuat UI isi Carousel yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, border atau outline setebal 1dp dengan warna Material 3 outline, warna Kartu atau “Card” berupa Material 3 Surface Container, dan memanggil “onDetailClick” jika ditekan. Di dalamnya, terdapat komponen “Row” dengan padding keseluruhan 16dp dan terdapat even spacing horizontal agar gambar berada di tengah. Komponen “Image” di dalam “Row” mengambil data dari DataSource.Kt, isi gambar memotong dirinya jika memenuhi View, dan modifier tinggi 200dp, lebar 400dp, dan lengkungan sebesar 16dp.

Composable “ItemCarousel()” berfungsi untuk memuat UI Carousel melalui komponen “LazyRow” yang menyimpan scroll state dan berhenti di item terdekat dalam scrolling. Di dalamnya, terdapat pemuatan data-data berdasarkan indeks ke dalam composable “CarouselCard()” beserta logika klik navigasi ke detail serta lebar child Composable mengambil penuh lebar View. Method preview “HomeScreenPreview()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

D. Essay

1. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

Jawab:

RecyclerView masih sering digunakan dibandingkan LazyColumn walaupun memiliki kode boilerplate dikarenakan dari aspek performa, RecyclerView lebih cepat dibandingkan LazyColumn. Hal ini disebabkan RecyclerView yang termasuk pada Android View based Code yang menerapkan Ahead Of Time compiled (AOT) dan sudah ter-include pada Sistem Operasi Android. Sedangkan, LazyColumn merupakan Library eksternal yang menerapkan Just In Time (JIT) compiled sehingga lebih lambat dibandingkan RecyclerView [1]. Selain itu, penggunaan RecyclerView lebih fleksibel dibandingkan LazyColumn [2].

2. Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut **pendapat kalian**, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

Jawab:

Ketika kita membuat suatu project, kita harus menerapkan Clean Architecture karena, menurut pendapat saya, Clean Architecture sangat menolong dalam membangun kode-kode yang lebih modular dan lebih bebas dari bug dan error, dibandingkan dengan project yang tidak menerapkan Clean Architecture. Tanpa penerapan Clean Architecture, project yang dibangun akan mulai terakumulasi dengan *technical debt*, yaitu suatu kondisi dimana perbaikan atau refaktor kode pada project lebih *costly* sepanjang project dikembangkan

E. Daftar Pustaka

- [1 StackOverFlow, “Jetpack Compose lazyrow considerably slower than a
] recyclerView on xml,” StackOverFlow, 24 Januari 2022. [Online]. Available:
<https://stackoverflow.com/questions/74563695/jetpack-compose-lazyrow-considerably-slower-than-a-recyclerview-on-xml>. [Diakses 1 Mei 2026].
- [2 Reddit, “Does LazyColumn in Compose really replace RecyclerView or am I
] missing something?,” Reddit, 19 Agustus 2021. [Online]. Available:
https://www.reddit.com/r/androiddev/comments/p7l8xy/does_lazycolumn_in_compose_really_replace/. [Diakses 1 Mei 2026].

Tautan GIT

Berikut adalah tautan GIT untuk Praktikum Pemrograman Mobile

<https://github.com/Arz-zrs/Pemrograman-Mobile>